EXP 14: PRO-CODE EXTENSION USING ZOHO CATALYST

AIM

To extend the low-code Library Book Issue Management application built in Zoho Creator with a pro-code (custom code) function developed and deployed on Zoho Catalyst, and to call that function from Zoho Creator.

PROCEDURE

1. **Understand the need for a pro-code extension**
   - Low-code is fast, but complex logic, heavy processing and special integrations can go beyond what forms, workflows and Deluge handle comfortably.
   - **Zoho Catalyst** is a serverless platform where developers write code (Node.js, Java or Python) as functions without managing servers.
   - Here, the fine calculation for overdue books is moved from Deluge to a Catalyst function.
2. **Create a Catalyst project**
   - Log in to the Zoho Catalyst console and create a new project, for example *LibraryExtension*.
   - The project provides the services needed to run the code, such as Functions, Data Store, File Store and logs.
3. **Set up the Catalyst CLI**
   - Install Node.js and then install the Catalyst command line tool, and log in to the Zoho account from the terminal.

```
npm install -g zcatalyst-cli
catalyst login
```

4. **Initialize the project and create the function**
   - Create a project folder, run the init command, select **Functions** and choose an **Advanced I/O** function with Node.js, so that it can be called as a REST API.

```
catalyst init
```

5. **Write the function code**
   - Write the logic that receives the number of overdue days and returns the fine amount (the daily rate used here is an example).

```
const express = require("express");
const app = express();
app.use(express.json());

app.post("/calculateFine", (req, res) => {
  const overdueDays = Number(req.body.overdue_days) || 0;
  const dailyRate = 2.0;                      // example rate per day
  const fine = overdueDays > 0 ? overdueDays * dailyRate : 0;
  res.status(200).json({ fine: fine });
});

module.exports = app;
```

6. **Test and deploy the function**
   - Test the function on the local system with the serve command, then deploy it to the Catalyst cloud.

```
catalyst serve
catalyst deploy
```

7. **Call the function from Zoho Creator**
   - Copy the function URL from the Catalyst console and call it from a Creator workflow or custom function using the Deluge **invokeUrl** task.
   - Store the URL and any keys in a connection or an application variable, not inside the script.

```
resp = invokeUrl
[
  url : "<catalyst-function-url>/calculateFine"
  type : POST
  parameters : {"overdue_days": overdueDays}.toString()
];
fineAmount = resp.get("fine");
```

8. **Test the integration and monitor**
   - Submit a Book Issue Request, check that the fine value returned by Catalyst is stored in the Fine Amount field, and review the Catalyst logs and the Creator execution logs.
   - Secure the function by allowing only authenticated calls and by checking the input values.

Zoho Catalyst Components Used

Component	Purpose	Use in this experiment
Functions (Advanced I/O)	Serverless code exposed as a REST API	calculateFine function
Catalyst CLI	Create, test and deploy projects from the terminal	init, serve and deploy commands
Logs	View execution output and errors	Check the function calls from Creator
Data Store (optional)	Relational database for custom data	Can store fine history if required

Low-Code and Pro-Code Together

Point	Zoho Creator (low-code)	Zoho Catalyst (pro-code)
Used for	Forms, reports, dashboards, workflows, roles	Custom logic, heavy processing, special integrations
Skills	Visual tools and basic Deluge	Programming in Node.js, Java or Python
Control	Limited to platform features	Full control over the code
Hosting	Managed by Creator	Serverless, managed by Catalyst

Advantages of the Pro-Code Extension

- Complex logic and heavy processing run outside Zoho Creator, so the Deluge scripts stay short and fast.
- The same Catalyst function can be reused by other applications and not only by Zoho Creator.
- Serverless hosting means there is no server to set up, patch or scale.
- Code can use libraries and services that are not available inside Deluge.

Points to Remember

- The Catalyst function has its own limits, logs and billing, which should be checked before use.
- Keep the function URL and keys in connections or variables and secure the endpoint.
- Handle errors in both the function and the Deluge script, so that a failed call does not stop the workflow.

RESULT

A pro-code function was created and deployed on Zoho Catalyst and connected to the Zoho Creator application using the Deluge invokeUrl task. This extended the Library Book Issue Management application beyond the limits of low-code, while keeping the fast development and easy maintenance of Zoho Creator.